

KOMPARANSI KINERJA ALGORITMA DIJKSTRA DAN A* DALAM MENENTUKAN JALUR KELUAR TERCEPAT (SHORTEST PATH PROBLEM) PADA KASUS LABIRIN SOLVER

¹Widya Louisa - 23031554180, ²Brillianda Annisaatulrohmah - 23031554216, ³Mutiara Khanza Asyifa - 23031554230

¹Program Studi S1 Sains Data, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Surabaya, Surabaya, 60231 Indonesia, email: widya.23180@mhs.unesa.ac.id

²Program Studi S1 Sains Data, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Surabaya, Surabaya, 60231 Indonesia, email: Briliyanda.23216@mhs.unesa.ac.id

³Program Studi S1 Sains Data, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Surabaya, Surabaya, 60231 Indonesia, email: mutiara.23230@mhs.unesa.ac.id

Abstract—Permasalahan jalur terpendek (*Shortest Path Problem*) merupakan salah satu fokus utama dalam studi matematika terapan dan ilmu komputer. Permasalahan ini bertujuan untuk menemukan jalur dengan total biaya terendah dari titik awal ke titik tujuan dalam sebuah graf berbobot. Algoritma Dijkstra dan A* adalah dua algoritma populer yang digunakan untuk menyelesaikan permasalahan ini. Keduanya memiliki tahap pengerjaan yang sederhana, namun kinerjanya mulai berbeda ketika diterapkan pada graf yang kompleks dan besar. Salah satu contoh graf yang kompleks adalah labirin, yang memiliki banyak simpul dan jalur potensial dengan pola yang tidak teratur. Dalam penelitian ini, algoritma Dijkstra dan A* diterapkan pada simulasi labirin untuk membandingkan kinerjanya dalam menentukan jalur terpendek. Algoritma Dijkstra mengevaluasi semua kemungkinan jalur secara menyeluruh tanpa mempertimbangkan arah menuju tujuan, sehingga cenderung kurang efisien pada graf besar. Sebaliknya, algoritma A* menggunakan fungsi heuristik berupa jarak Manhattan untuk memandu pencarian jalur secara terarah ke titik tujuan, sehingga prosesnya lebih cepat dan efisien. Hasil penelitian menunjukkan bahwa algoritma A* memiliki kinerja yang lebih baik dibandingkan algoritma Dijkstra dalam menentukan jalur terpendek pada labirin. Hal ini disebabkan oleh kemampuan algoritma A* dalam memprioritaskan simpul yang paling relevan melalui kombinasi biaya aktual dan estimasi biaya ke tujuan. Penelitian ini memberikan wawasan tentang efisiensi kedua algoritma dalam menyelesaikan permasalahan jalur terpendek pada graf yang kompleks.

Keywords—*Shortest path, Labirin, Dijkstra, A**

I. PENDAHULUAN

Perkembangan teknologi yang sangat cepat harus didukung dengan perkembangan dan implementasi algoritma yang mangkus, sehingga teknologi tersebut dapat memberikan hasil yang baik, efektif dan efisien.

Pada saat ini, banyak algoritma yang berkembang untuk mencapai solusi dari sebuah permasalahan. Namun, algoritma tersebut belum tentu memberikan solusi yang optimal terhadap masalah yang ada. Solusi yang optimal dapat dihasilkan dengan menambahkan sebuah informasi tambahan pada algoritma tersebut yang akan digunakan oleh algoritma sebagai dasar dalam pencarian solusi.

Permasalahan jalur terpendek (*Shortest Path Problem*) menjadi salah satu fokus utama dalam matematika terapan dan ilmu komputer, terutama karena penerapannya yang luas pada navigasi, desain jaringan, dan permainan komputer. Di antara berbagai algoritma pencarian jalur terpendek, algoritma Dijkstra dan A* (*A-Star*) merupakan yang paling sering digunakan karena efisiensi dan kesederhanaan implementasinya. Dijkstra pertama kali diperkenalkan oleh Edsger Dijkstra pada tahun 1956 dan dikenal sebagai algoritma deterministik yang dapat menjamin solusi optimal pada graf berbobot. Algoritma A*, yang dikembangkan oleh Hart, Nilsson, dan Raphael pada tahun 1968, adalah pengembangan dari Dijkstra dengan penambahan heuristik untuk memandu jalur pencarian lebih efisien (Hart, Nilsson, dan Raphael, 1972). Kedua algoritma ini memiliki pendekatan yang mirip, tetapi perbedaan utama pada A* terletak pada penggunaan heuristik untuk mempercepat pencarian, khususnya pada graf kompleks.

Labirin adalah salah satu bentuk graf kompleks yang terdiri dari banyak cabang, simpul, dan jalur potensial. Hal ini membuat labirin menjadi studi kasus ideal untuk mengevaluasi kinerja kedua algoritma tersebut. Prinsip dari permainan labirin adalah menemukan jalur keluar dari titik awal ke titik

tujuan melalui jalan yang penuh dengan cabang dan jalur buntu (H, 2011). Kompleksitas graf labirin sering kali membuat algoritma seperti Dijkstra lebih lambat karena harus mengevaluasi semua jalur, sedangkan A* lebih unggul dengan memanfaatkan heuristik untuk memperkirakan jalur ke tujuan (Nugraha, 2018).

Penelitian ini bertujuan untuk membandingkan kinerja algoritma Dijkstra dan A* dalam menemukan jalur terpendek pada labirin yang dihasilkan secara dinamis menggunakan algoritma Prim yang dimodifikasi. Visualisasi proses pencarian jalur dilakukan menggunakan Python dengan pustaka Pygame untuk menunjukkan langkah-langkah pencarian secara real-time. Fokus utama dari penelitian ini adalah mengevaluasi waktu eksekusi dan space complexity dari kedua algoritma, yang menjadi indikator efisiensi pencarian jalur pada graf kompleks seperti labirin.

Hasil penelitian menunjukkan bahwa meskipun algoritma Dijkstra memberikan solusi optimal di semua kasus, algoritma A* memiliki keunggulan dalam hal waktu eksekusi. Hal ini dikarenakan A* menggunakan heuristik berupa Jarak Manhattan untuk memandu pencarian lebih langsung ke tujuan, sehingga mengurangi jumlah simpul yang perlu dievaluasi. Dengan pendekatan ini, penelitian memberikan wawasan tentang bagaimana kedua algoritma bekerja dalam konteks graf kompleks.

Rumusan Masalah:

1. Bagaimana kinerja algoritma Dijkstra dan A* dalam menentukan jalur terpendek pada graf kompleks seperti labirin, khususnya dari segi waktu eksekusi?

II. DASAR TEORI

Proyek ini berfokus pada permasalahan jalur terpendek (Shortest Path Problem), yaitu menemukan rute paling efisien dari titik awal ke tujuan pada graf berbentuk labirin. Labirin dipilih sebagai studi kasus karena sifatnya yang kompleks dengan banyak simpul dan jalur bercabang. Untuk menyelesaikan masalah ini, proyek menggunakan algoritma Prim yang dimodifikasi untuk menghasilkan labirin, sedangkan algoritma Dijkstra dan A* digunakan untuk mencari jalur terpendek. Teori dasar dari algoritma-algoritma ini sangat penting untuk memahami pendekatan dan keunggulannya masing-masing, terutama dalam konteks graf yang kompleks. Bagian ini menjelaskan permasalahan yang diangkat, serta teori terkait algoritma Dijkstra dan A* yang menjadi inti dari project ini.

A. Shortest Path Problem dengan Labirin Solver

Permasalahan jalur terpendek (Shortest Path Problem) merupakan salah satu jenis masalah optimisasi dalam ilmu komputer dan matematika terapan. Tujuannya adalah

menemukan rute dengan bobot terkecil dari satu titik awal ke titik tujuan pada sebuah graf. Masalah ini sering muncul dalam berbagai aplikasi nyata, seperti perencanaan rute transportasi, navigasi peta, desain jaringan komputer, dan bahkan dalam permainan.

Dalam Project ini, labirin digunakan sebagai studi kasus karena sifatnya yang kompleks dengan banyak simpul (nodes) dan jalur potensial (edges). Labirin direpresentasikan sebagai graf berbentuk grid, dimana setiap simpul merepresentasikan sebuah sel, dan setiap jalur merepresentasikan koneksi antar sel. Sifat kompleksitas labirin, seperti banyaknya cabang dan kemungkinan jalur, membuatnya ideal untuk menguji performa algoritma pencarian jalur terpendek. Project ini memanfaatkan algoritma Dijkstra dan A*, dua algoritma optimasi terkenal, untuk menyelesaikan permasalahan pencarian jalur pada labirin, serta menggunakan algoritma prim yang dimodifikasi untuk menghasilkan labirin sebagai graf kompleks.

A. Algoritma Dijkstra

Algoritma Dijkstra merupakan salah satu algoritma yang sering digunakan dalam pencarian rute terpendek karena mudah digunakan. Dalam mencari solusi, algoritma Dijkstra menggunakan prinsip greedy, yaitu untuk menentukan solusi optimasi pada setiap langkahnya agar bisa mendapatkan solusi optimasi untuk Langkah selanjutnya yang mengarah pada pilihan terbaik. Hal ini membuat kompleksitas waktu eksekusi algoritma Dijkstra menjadi cukup besar, yaitu $(v * \log(v + e))$ di mana v adalah *node* dan e adalah sisi yang menghubungkan *node* (Risald et al., 2017).

Algoritma Dijkstra selalu menghitung seberapa jauh proses traversal setiap mencapai suatu node. Algoritma ini juga membangun jalur minimal dari node awal (s) ke node lainnya (v), mengeksplor setiap node yang berdekatan hingga menemukan rute terpendek menuju node tujuan (t). Proses eksplorasi ini dikenal sebagai *edge relaxation* (Ortega-Arranz et al., 2014).

Dijkstra bekerja mirip dengan BFS, yaitu dengan menggunakan prinsip *queue*. Tetapi yang digunakan pada Dijkstra adalah Queue prioritas sehingga satu-satunya node yang memiliki prioritas tertinggi akan dicari. Dalam menentukan node prioritas, algoritma ini membandingkan setiap nilai dari setiap node yang disimpan untuk dibandingkan dengan nilai yang ditemukan dari rute berikutnya dan begitu seterusnya hingga node tujuan ditemukan (Wang et al., 2015). Sama seperti BFS, pada algoritma ini juga diberikan warna node berbeda sesuai dengan keterangannya masing-masing untuk melacak proses eksplorasi.

B. Algoritma A*

Algoritma A* adalah algoritma yang dikemukakan oleh Hart, Nilsson, dan Raphael pada tahun 1968. Algoritma A* merupakan salah satu algoritma Branch & Bound atau disebut juga sebagai sebuah algoritma untuk melakukan pencarian solusi dengan menggunakan informasi tambahan (heuristik) dalam menghasilkan solusi yang optimal.

Algoritma A* merupakan algoritma yang digunakan untuk menentukan rute terpendek objek menuju ke tujuan, dengan menghitung harga yang harus dipakai untuk mencari harga terkecil yang harus dibayarkan. Algoritma diperlukan untuk membuat game lebih menarik. Hal ini berbanding lurus dengan jumlah pengguna, semakin menarik game yang dimainkan, semakin banyak orang yang akan memainkan game tersebut. Di dalam sebuah game algoritma biasa digunakan untuk menentukan level, tingkat kesulitan, skor/nilai, hingga sebagai pengambilan keputusan (Oktanugraha & Nudin, 2020). Algoritma A* termasuk dalam kategori pencarian *informed search*, karena memanfaatkan informasi tambahan berupa heuristik untuk membimbing jalur pencarian secara efisien menuju tujuan (Ahmad & Widodo, 2017).

Algoritma A* menggunakan estimasi jarak terdekat (cost/jarak sebenarnya) untuk mencapai tujuan (goal) dan memiliki nilai heuristik yang digunakan sebagai dasar pertimbangan pemilihan jalur (Deviana, 2015). Algoritma A* ini memeriksa node dengan menggabungkan $g(n)$, yaitu cost yang dibutuhkan untuk mencapai sebuah node dan $h(n)$ yaitu cost yang didapat dari node ke tujuan (Dewa, 2016). Sehingga dapat dirumuskan sebagai berikut (Erniyati & Mulyati, 2019) :

$$f(n) = g(n) + h(n)$$

$f(n)$ = perkiraan perkiraan total cost terendah dari setiap path yang akan dilalui dari node n ke node tujuan

$g(n)$ = biaya yang sudah dikeluarkan dari keadaan awal sampai keadaan n

$h(n)$ = perkiraan heuristic atau cost atau path dari node n ke tujuan.

Jarak Manhattan adalah cara untuk mengukur jarak antara dua titik dalam ruang dua dimensi. Namanya diambil dari jaringan jalan di Kota New York, di mana jarak antara dua titik dihitung sebagai jumlah total perpindahan horizontal dan vertikal yang diperlukan untuk mencapai tujuan, tanpa memperhitungkan pergerakan diagonal. Secara matematis, Jarak Manhattan antara dua titik (x_1, y_1) dan (x_2, y_2) dalam ruang koordinat dua dimensi didefinisikan sebagai $|x_1 - x_2| + |y_1 - y_2|$. Metrik ini sering digunakan dalam berbagai bidang,

seperti komputasi graf, perencanaan rute, dan pemrosesan gambar, karena sifatnya yang

III. Implementasi

Bagian ini menjelaskan langkah-langkah yang dilakukan dalam mendesain dan menganalisis dua algoritma pencarian jalur terpendek, yaitu Dijkstra dan A*, dalam konteks menyelesaikan masalah labirin (shortest path problem). Proyek ini bertujuan untuk mengevaluasi dan membandingkan kinerja kedua algoritma dalam menemukan jalur keluar tercepat dari labirin yang direpresentasikan sebagai graf berbentuk grid. Deskripsi berikut akan menguraikan logika di balik kedua algoritma, memberikan pseudocode untuk implementasi, serta menganalisis kompleksitas waktu dan ruang dari masing-masing algoritma.

A. Implementasi Algoritma I

- Algoritma Dijkstra digunakan untuk mencari jalur terpendek dari titik awal ke titik tujuan pada graf berbobot dengan pendekatan greedy. Pada labirin yang dihasilkan menggunakan algoritma Prim yang dimodifikasi, graf dianggap sebagai grid dua dimensi di mana setiap simpul mewakili sel, dan setiap jalur mewakili koneksi antar sel.
- Dalam implementasi Dijkstra, algoritma ini mengelola antrian prioritas untuk memilih simpul dengan jarak terpendek yang belum dikunjungi. Proses iteratif dilakukan hingga mencapai tujuan atau mengeksplorasi seluruh graf. Pseudocode untuk implementasi Dijkstra adalah sebagai berikut:

```
ALGORITMA Dijkstra(mulai, tujuan):
    # Inisialisasi
    BUAT antrian prioritas kosong
    mulai.jarak = 0
    MASUKKAN mulai ke antrian prioritas dengan prioritas 0

    SELAMA antrian prioritas TIDAK KOSONG:
        simpul_saat_ini = AMBIL simpul dengan jarak terkecil dari antrian

        JIKA simpul_saat_ini == tujuan:
            HENTIKAN pencarian

        UNTUK SETIAP tetangga dari simpul_saat_ini:
            # Hitung jarak alternatif
            jarak_alternatif = simpul_saat_ini.jarak + 1

            JIKA jarak_alternatif < tetangga.jarak ATAU tetangga.jarak BELUM TERDEFINISI:
                tetangga.jarak = jarak_alternatif
                tetangga.parent = simpul_saat_ini
                MASUKKAN tetangga ke antrian prioritas dengan prioritas jarak_alternatif

    KEMBALIKAN jalur dari mulai ke tujuan
```

Gambar 1. Visualisasi Pseudocode Algoritma Dijkstra

- Kompleksitas Waktu dan Ruang:
 - Kompleksitas waktu dari algoritma Dijkstra adalah $O((v+e)\log v)$, di mana v adalah jumlah simpul (nodes) dan e adalah jumlah sisi yang menghubungkan nodes (edges) dalam graf.

- Kompleksitas ruang adalah $O(v)$, karena algoritma ini menyimpan jarak dari setiap simpul dan antrian prioritas.

B. Implementasi Algoritma 2

- Algoritma A* merupakan pengembangan dari Dijkstra dengan penambahan fungsi heuristik yang digunakan untuk memperkirakan biaya menuju tujuan. Dalam implementasi A*, setiap simpul memiliki dua nilai: $g(n)$, yaitu biaya dari simpul awal ke simpul n , dan $h(n)$, yaitu estimasi biaya dari simpul n ke tujuan. Fungsi $f(n)$ adalah jumlah dari $g(n)$ dan $h(n)$, yang akan digunakan untuk memprioritaskan simpul yang akan dieksplorasi.
- Pseudocode untuk implementasi A* adalah sebagai berikut:

```
ALGORITMA A_Star(mulai, tujuan):
    # Inisialisasi
    BUAT antrian prioritas kosong
    mulai.g = 0
    mulai.h = HITUNG_HEURISTIK(mulai, tujuan)
    mulai.f = mulai.g + mulai.h
    MASUKKAN mulai ke antrian prioritas dengan prioritas mulai.f

    SELAMA antrian prioritas TIDAK KOSONG:
        simpul_saat_ini = AMBIL simpul dengan f-score terkecil dari antrian

        JIKA simpul_saat_ini == tujuan:
            HENTIKAN pencarian

        UNTUK SETIAP tetangga dari simpul_saat_ini:
            # Hitung g-score sementara
            g_sementara = simpul_saat_ini.g + 1

            JIKA g_sementara < tetangga.g ATAU tetangga.g BELUM TERDEFINISI:
                tetangga.parent = simpul_saat_ini
                tetangga.g = g_sementara
                tetangga.h = HITUNG_HEURISTIK(tetangga, tujuan)
                tetangga.f = tetangga.g + tetangga.h
                MASUKKAN tetangga ke antrian prioritas dengan prioritas tetangga.f

    KEMBALIKAN jalur dari mulai ke tujuan

FUNGSI HITUNG_HEURISTIK(simpul1, simpul2):
    RETURN jarak Manhattan = |simpul1.x - simpul2.x| + |simpul1.y - simpul2.y|
```

Gambar 2. Visualisasi Pseudocode Algoritma A*

- Kompleksitas Waktu dan Ruang:
 - Kompleksitas waktu dari A* bergantung pada heuristik yang digunakan. Dalam kasus terburuk, kompleksitas waktu adalah $O(b^m)$, di mana b adalah rata-rata jumlah anak setiap simpul dan m adalah kedalaman solusi. Namun, jika heuristik mendekati optimal, A* dapat lebih efisien.
 - Kompleksitas ruang adalah $O(b^m)$, karena algoritma ini menyimpan semua simpul yang dieksplorasi dalam memori.

C. Perbandingan Kompleksitas Algoritma 1 dan Algoritma 2

Kedua algoritma ini memiliki pendekatan yang berbeda, yang mempengaruhi kompleksitas dan kinerja mereka dalam memecahkan masalah labirin.

- Dijkstra, tanpa heuristik, mengeksplorasi semua kemungkinan jalur hingga menemukan jalur

terpendek, yang menjadikannya lebih lambat pada graf besar atau graf dengan banyak cabang. Di sisi lain, A* menggunakan heuristik untuk mempercepat pencarian, sehingga sering kali lebih efisien dalam hal waktu pemrosesan dan jumlah simpul yang dieksplorasi, terutama pada graf yang kompleks seperti labirin.

- Algoritma Dijkstra cenderung lebih stabil dalam hal kompleksitas waktu, tetapi mungkin memerlukan lebih banyak waktu untuk mengeksplorasi graf besar.
- Algoritma A* lebih cepat pada graf dengan banyak cabang jika heuristik yang digunakan akurat, tetapi dapat menjadi lebih mahal jika heuristiknya buruk atau tidak ada.
- Secara keseluruhan, A* sering kali lebih efisien pada graf dengan banyak cabang dan jalur yang lebih kompleks karena penggunaan heuristik yang cerdas, sementara Dijkstra lebih sederhana dan lebih andal pada graf dengan struktur yang lebih sederhana atau apabila heuristik tidak dapat diterapkan.

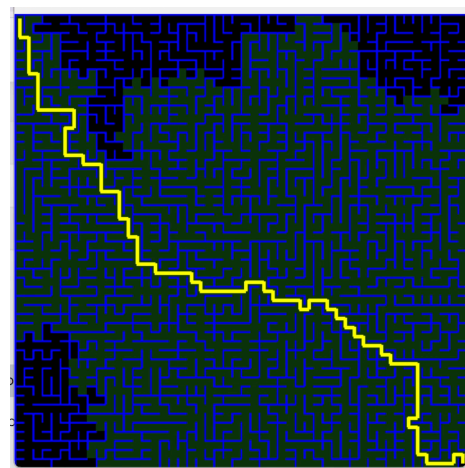
III. HASIL DAN DISKUSI

Dari hasil implementasi dan pengujian algoritma Dijkstra dan A* menunjukkan performa yang berbeda dalam menyelesaikan masalah labirin (shortest path problem).

A. Hasil Implementasi Algoritma

- Algoritma Dijkstra

Hasil algoritma Dijkstra menunjukkan bahwa algoritma ini mampu menemukan jalur terpendek pada labirin yang di uji meskipun dengan waktu yang relatif lama. Berikut merupakan hasil implementasinya:



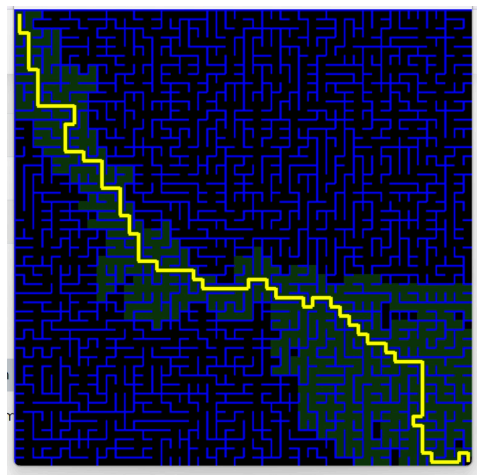
Gambar 1. Visualisasi Algoritma Dijkstra

Kompleksitas Waktu dan Ruang:

- Dijkstra's Algorithm completed in 34.1995 seconds
- Space complexity:
 - Peak priority queue size: 0.68 KB
 - Visited surface size: 0.06 KB
 - Total memory usage: 0.74 KB
- Number of nodes explored: 2083
- Path length: 108

• Algoritma A*

Hasil dari algoritma A* lebih efisien dalam menemukan jalur terpendek. Waktu yang dibutuhkan untuk eksekusinya pun lebih singkat dibandingkan algoritma Dijkstra. Berikut adalah hasil implementasinya menggunakan labirin yang sama:



Gambar 2. Visualisasi Algoritma A*

Kompleksitas Waktu dan Ruang:

- A* Algorithm completed in 11.0637 seconds
- Space complexity:
 - Peak open set size: 0.68 KB
 - Visited surface size: 0.06 KB
 - Total memory usage: 0.74 KB
- Number of nodes explored: 670
- Path length: 108

B. Kelebihan dan Kekurangan Algoritma

Tabel 1. Perbandingan Kelebihan dan Kekurangan Algoritma

Aspek	Algoritma Dijkstra	Algoritma A*
Kelebihan	Dijkstra selalu memberikan solusi jalur terpendek pada semua jenis	unggul dalam efisiensi waktu karena menggunakan

Aspek	Algoritma Dijkstra	Algoritma A*
	graf berbobot, terlepas dari strukturnya. tidak memerlukan fungsi heuristik, sehingga selalu dapat diterapkan pada semua graf tanpa tambahan informasi. Algoritma ini lebih mudah diimplementasikan dan tidak rentan terhadap kesalahan estimasi biaya. Tidak ada variasi kinerja yang signifikan karena Dijkstra mengevaluasi semua jalur	fungsi heuristik, seperti jarak Manhattan, untuk memandu pencarian secara lebih terarah menuju tujuan. Hal ini membuat A* lebih cepat dalam mengevaluasi simpul pada graf besar atau kompleks, terutama jika heuristiknya admissible (tidak melebihi-lebihkan biaya). Selain itu, A* fleksibel karena dapat disesuaikan dengan berbagai fungsi heuristik sesuai kebutuhan.
Kekurangan	Dijkstra mengevaluasi semua simpul yang terhubung tanpa mempertimbangkan arah ke tujuan, sehingga banyak waktu yang terbuang untuk mengeksplorasi jalur yang tidak relevan.	ketergantungannya terhadap heuristik; jika heuristiknya buruk atau tidak akurat, efisiensinya menurun. A* juga memiliki kompleksitas memori tinggi karena menyimpan semua simpul yang dievaluasi dalam proses pencarian.

C. Hubungan dengan Teori

Dilihat dari hasil pengujian Algoritma Dijkstra dan A* , hasil mendukung teori dasar dari kedua algoritma. Algoritma Dijkstra, yang bekerja secara Greedy pada semua simpul, menjamin optimalitas jalur terpendek namun kurang efisien dalam eksplorasi. Di sisi lain, A* memanfaatkan pendekatan "best-first" yang didukung heuristik, sehingga lebih efisien dalam banyak kasus.

D. Penjelasan Performa Algoritma

Dilihat dari efisiensi waktu dan langkahnya Algoritma A* lebih efisien dibandingkan Dijkstra.

1. Algoritma Dijkstra: Algoritma Dijkstra selalu memberikan hasil jalur terpendek dengan cara mengeksplorasi semua simpul yang memiliki jarak minimum, hal itu membuat Algoritma ini tidak efisien dalam

waktunya. Selain itu eksplorasi menyeluruh ini dapat menjadi tidak efisien pada graf yang sangat besar karena tidak memiliki mekanisme untuk memprioritaskan simpul yang lebih relevan.

2. Algoritma A*: Algoritma A* menunjukkan efisiensi lebih tinggi dibandingkan Dijkstra karena memanfaatkan kombinasi fungsi jarak aktual dan heuristik untuk memprioritaskan simpul yang lebih dekat ke tujuan. Dengan demikian, jumlah simpul yang dieksplorasi lebih sedikit, yang berarti waktu komputasi lebih rendah.

E. Aplikasi dalam Kasus Nyata

1. Algoritma Dijkstra: Penerapan pada dunia nyata adalah misalnya routing pada peta kota karena informasi heuristik tidak tersedia. Contohnya adalah perhitungan rute optimal dalam jaringan transportasi umum (tanpa memperhatikan kondisi lalu lintas real-time).
2. Algoritma A*: Penerapan algoritma ini pada dunia nyata cocok untuk navigasi real-time, contohnya adalah GPS karena memperhitungkan jarak aktual dan estimasi waktu perjalanan berdasarkan kondisi lalu lintas.

F. Kesimpulan

Dari hasil percobaan dan pengamatan dapat disimpulkan bahwa Algoritma A* memiliki efisiensi waktu yang lebih baik dibandingkan Algoritma Dijkstra. Langkah yang diambil A* pada setiap eksekusi juga menunjukkan hasil yang lebih sedikit dibandingkan Algoritma Dijkstra. Namun untuk pencarian jalur tercepat kedua Algoritma ini memiliki efisiensi solusi yang sama.

VIDEO LINK AT YOUTUBE (Heading 5)

<https://youtu.be/w4qgTwx4QPI?feature=shared>

CODE

<https://drive.google.com/drive/folders/1e0FUVdJfjo9RQBHVBRfW-iU0W5SMlZ?usp=sharing>

REFERENSI

- [1] G. Altintas, "Exact Solution of the Shortest Path in a Maze Based on Mechanical Analogies and Considerations". Manisa Celal Bayar University, 2020. <https://www.researchgate.net/publication/343480719>
- [2] A. Islam, F. Ahmad, and P. Sathya, "Shortest Distance Maze Solving Robot," *International Journal of Research in Engineering and Technology*, vol. 5, no. 7, pp. 253-259, Jul. 2016. <http://ijret.esatjournals.org>
- [3] S. Paul and C. B. C. Latha, "Shortest Path Traversal in a Maze with Wall Following Robot," *AIP Conference Proceedings*, vol. 2670, no. 030002, pp. 1-7, Dec. 2022. <https://doi.org/10.1063/5.0116117>
- [4] D. Rachmawati and L. Gustin, "Analysis of Dijkstra's Algorithm and A* Algorithm in Shortest Path Problem," *Journal of Physics: Conference Series*, vol. 1566, no. 1, p. 012061, 2020. <https://doi.org/10.1088/1742-6596/1566/1/012061>
- [5] I. A. Wulandari and P. Sukmasetyan, "Implementasi Algoritma Dijkstra untuk Menentukan Rute Terpendek Menuju Pelayanan Kesehatan," *Jurnal Ilmiah Sistem Informasi*, vol. 1, no. 1, pp. 30-37, Mar. 2022.
- [6] A. C. Prasetyo, M. P. Arnandi, H. S. Hudnanto, and B. Setiaji, "Perbandingan Algoritma Astar dan Dijkstra Dalam Menentukan Rute Terdekat," *Jurnal Ilmiah SISFOTENIKA*, vol. 9, no. 1, pp. 36-46, Jan. 2019.
- [7] D. Y. A. Falloa and V. R. Bulub, "Penerapan Algoritma A Star (A*) pada Game Labirin," *Jurnal Pendidikan Teknologi Informasi (JUKANTI)*, vol. 5, no. 1, pp. 118-124, Apr. 2022.
- [8] H. Tilawah, "Penerapan Algoritma A-star (A*) Untuk Menyelesaikan Masalah Maze," *Makalah IF3051 Strategi Algoritma*, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, Bandung, Indonesia, Dec. 2011.
- [9] M. F. Irhab, "Optimalisasi pencarian jalur dalam labirin menggunakan algoritma A*," *Makalah IF2211 Strategi Algoritma*, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, Bandung, Indonesia, June 2024.
- [10] M. I. Rahajeng, "Perbandingan perbedaan heuristik pada algoritma A* dalam penyelesaian labirin," *Makalah IF2211 Strategi Algoritma*, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, Bandung, Indonesia, Apr. 2019.